

EX NO : 8

DATE : 17-07-26

## LOGISTIC REGRESSION

### AIM:

To implement **Logistic Regression** on the Titanic dataset for predicting passenger survival, evaluate the model performance, and plot the training and testing accuracy.

### PROBLEM STATEMENT:

The Titanic dataset contains details of passengers such as age, gender, ticket class, fare, etc. The objective is to build a **Logistic Regression classification model** that predicts whether a passenger survived (1) or not (0). The model is evaluated using accuracy, confusion matrix, and classification report. Finally, the training and testing accuracy are compared using a graph.

### DATASET: Titanic

The Titanic dataset contains information about passengers aboard the RMS Titanic.

- **Number of Records:** 891
- **Target Variable:** survived
  - 0 → Did Not Survive
  - 1 → Survived

### CODE:

```
# Import libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load Dataset
df = sns.load_dataset('titanic')

print("First 5 rows")
print(df.head())

print("\nDataset Shape:", df.shape)

# Select useful features
df = df[['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked']]

# Handle Missing Values
df['age'] = df['age'].fillna(df['age'].median())
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])

# Convert Categorical Columns
encoder = LabelEncoder()
```

```

df['sex'] = encoder.fit_transform(df['sex'])
df['embarked'] = encoder.fit_transform(df['embarked'])

# Split Features and Target
X = df.drop('survived', axis=1)
y = df['survived']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Train Logistic Regression Model
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Accuracy
train_acc = model.score(X_train, y_train)
test_acc = model.score(X_test, y_test)

print("\nTraining Accuracy :", round(train_acc*100,2),"%")
print("Testing Accuracy :", round(test_acc*100,2),"%")

# Confusion Matrix
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report")
print(classification_report(y_test, y_pred))

# Accuracy Graph
accuracy = [train_acc*100, test_acc*100]
labels = ['Training Accuracy', 'Testing Accuracy']

plt.figure(figsize=(6,5))
plt.bar(labels, accuracy)

plt.ylabel("Accuracy (%)")
plt.title("Training vs Testing Accuracy")
plt.ylim(0,100)

for i,v in enumerate(accuracy):
    plt.text(i, v+1, f'{v:.2f}%', ha='center', fontsize=11)

plt.show()

```

## OUTPUT:

Dataset Shape: (891, 15)

Training Accuracy : 80.06 %

Testing Accuracy : 81.01 %

Confusion Matrix

```
[[90 15]
 [19 55]]
```

Classification Report

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.83      | 0.86   | 0.84     | 105     |
| 1            | 0.79      | 0.74   | 0.76     | 74      |
| accuracy     |           |        | 0.81     | 179     |
| macro avg    | 0.81      | 0.80   | 0.80     | 179     |
| weighted avg | 0.81      | 0.81   | 0.81     | 179     |

